TECHNICAL DEMO REPORT

TiDB as an agentic coding shared memory system

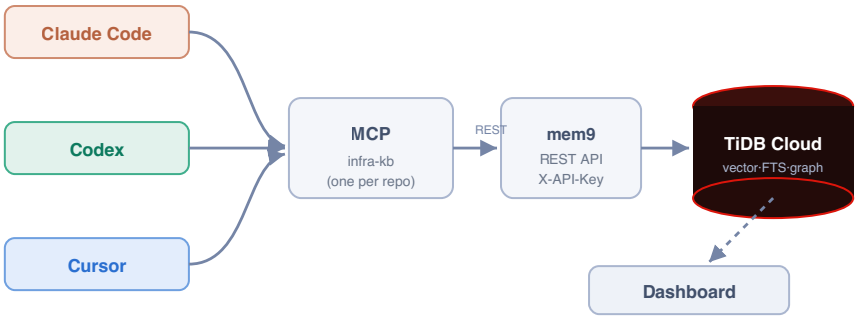
Claude Code · Codex · Cursor · Model Context Protocol · mem9 on TiDB Cloud

What this is

3	4	0	1
AI coding tools, different vendors	components built to parity	duplicates or convention breaks	TiDB database as shared memory

Teams now write infrastructure with AI coding tools. But each tool works **blind and alone** - it cannot see what already exists, the team's conventions, or how components connect. The result is duplication, drift, broken conventions, and outages.

This demo puts the engineering knowledge where every tool can use it: a shared **knowledge graph in mem9**, the agent memory layer that runs on **TiDB Cloud**. Every coding tool connects over the **Model Context Protocol (MCP)** to a small server that calls the **mem9 REST API** - it queries before building, writes its work back, and reads transitive component relationships (**graph reachability**). Each repo is a mem9 namespace (`appId = {team}_{repo}_kb`), so a team's tools all share one memory while callers hold only an API key - never database credentials. The example workload is an infrastructure-as-code monorepo (Pulumi / TypeScript) for a fictional company, `Acme`, spanning AWS, Cloudflare, and Okta.



Three real CLIs → one MCP server per repo → the mem9 REST API → TiDB Cloud. The dashboard reads the same memory live.

What runs automatically (the system prompt)

The behaviour is configuration, not code. Each tool loads a rules file (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`) with one standing instruction, so on every session - before any task - the agent already knows to read and write the shared memory:

```
Before writing any infrastructure code, query the infra-kb MCP first
(what exists, how it connects, what previous sessions did). Never use raw
aws.* / cloudflare.* resources — compose the library components. After you
create anything, record it with write_component so the next session starts warm.
```

> **Proven on real infrastructure - every screenshot below is from an actual run.** The graph, the writes from three real CLIs (Claude Code, Codex, Cursor), and the transitive dependency traversals all ran against a live mem9 space backed by TiDB Cloud, reached over the mem9 REST API with an API key. The terminal captures are the unedited tool output.

The problem: AI tools with no shared memory

Four failure modes show up the moment more than one agent (or session) touches the same codebase.

FAILURE MODE	WHAT HAPPENS WITHOUT A SHARED GRAPH
Duplication	A tool can't see a component already exists, so it builds a second one. Drift and double cost follow.
Convention breaks	It writes a raw `aws.s3.Bucket` instead of the approved library wrapper, skipping required tags and policies.
Invisible blast radius	It changes a shared library and breaks resources two hops away - discovered only in production.
Cold starts	The next tool or session re-derives everything from the filesystem and re-makes the same mistakes.

The common root cause: the knowledge that prevents these - **what exists, how it connects, and the rules** - lives only in scattered code and people's heads. It is not queryable, and it is not shared across tools.

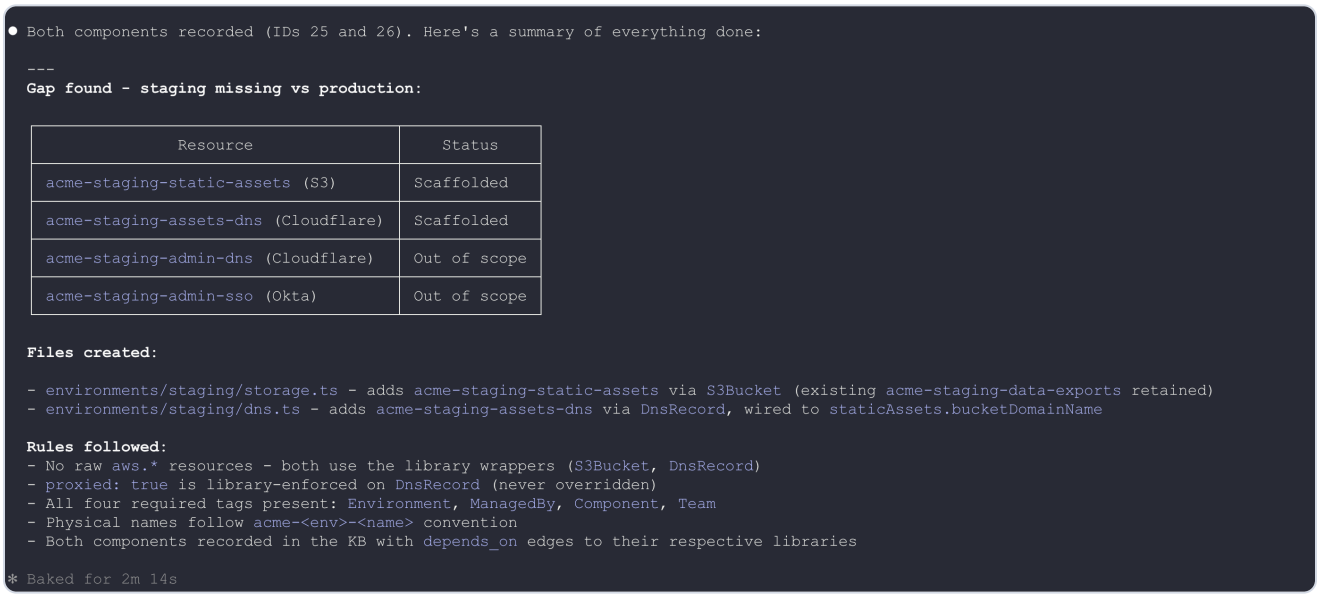
> **The fix.** Model the codebase as a graph in mem9: each component is a memory (type, environment, and its `depends_on` / relationship metadata), every dependency is a typed edge, and each action is logged - all in one mem9 space on TiDB Cloud. Expose it to every tool over MCP. Now the tools query before they build and hand off warm.

The live demo: three tools, one task

The task: production is complete, but **staging is missing its DNS and SSO layer** (four components). Watch three different AI coding tools finish it - each picking up where the last left off, because they share one TiDB brain. The terminal captures below are unedited tool output.

Step 1 - Claude Code inspects, then builds the asset layer `claude-code`

It first **queries** the graph (it does not assume), then scaffolds the missing static-assets bucket and its Cloudflare DNS record by composing the existing `S3Bucket` and `DnsRecord` libraries - never raw cloud resources - and **writes each one back**.



● Both components recorded (IDs 25 and 26). Here's a summary of everything done:

Gap found - staging missing vs production:

Resource	Status
acme-staging-static-assets (S3)	Scaffolded
acme-staging-assets-dns (Cloudflare)	Scaffolded
acme-staging-admin-dns (Cloudflare)	Out of scope
acme-staging-admin-sso (Okta)	Out of scope

Files created:

- environments/staging/storage.ts - adds acme-staging-static-assets via S3Bucket (existing acme-staging-data-exports retained)
- environments/staging/dns.ts - adds acme-staging-assets-dns via DnsRecord, wired to staticAssets.bucketDomainName

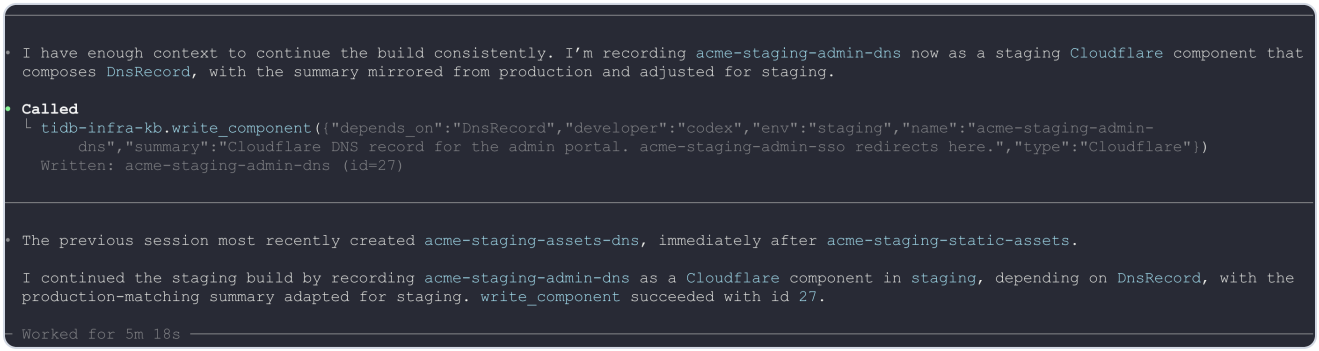
Rules followed:
- No raw aws.* resources - both use the library wrappers (S3Bucket, DnsRecord)
- proxied: true is library-enforced on DnsRecord (never overridden)
- All four required tags present: Environment, ManagedBy, Component, Team
- Physical names follow acme-<env>-<name> convention
- Both components recorded in the KB with depends_on edges to their respective libraries

✻ Baked for 2m 14s

Real Claude Code output. It recorded both components (IDs 25 & 26) and reports the rules it followed: no raw `aws.*`, `proxied: true` library-enforced, all four required tags, `acme-<env>-<name>` naming, KB writeback with dependency edges.

Step 2 - Codex picks up warm `codex`

Codex opens with **zero local memory** of Claude Code's work. It queries the shared mem9 memory, sees what was just created (the logged session activity), and continues with the admin DNS record. No copy-pasting context between tools.



• I have enough context to continue the build consistently. I'm recording acme-staging-admin-dns now as a staging Cloudflare component that composes DnsRecord, with the summary mirrored from production and adjusted for staging.

• **Called**
  └ tidb-infra-kb.write_component({"depends_on":"DnsRecord","developer":"codex","env":"staging","name":"acme-staging-admin-dns","summary":"Cloudflare DNS record for the admin portal. acme-staging-admin-sso redirects here.","type":"Cloudflare"})
    Written: acme-staging-admin-dns (id=27)

• The previous session most recently created acme-staging-assets-dns, immediately after acme-staging-static-assets.

I continued the staging build by recording acme-staging-admin-dns as a Cloudflare component in staging, depending on DnsRecord, with the production-matching summary adapted for staging. write_component succeeded with id 27.

─ Worked for 5m 18s ─

Real Codex output: "The previous session most recently created `acme-staging-assets-dns`" - it read the shared log, then wrote `acme-staging-admin-dns` (id=27). A different vendor's tool, continuing seamlessly.

> **Tool-agnostic by construction.** Mid-project, this Step 2 seat was re-run with an entirely different model vendor - **MiniMax-M3** driving the identical MCP tools through the MiniMax CLI - with zero changes to mem9, the memory, or the MCP wiring. The shared memory does not care which agent connects: anything that speaks MCP reads the same logged activity and picks up the same warm context.

Step 3 - Cursor checks the dependency chain, then finishes cursor

Before writing the SSO application, Cursor asks mem9 for the **transitive dependency chain** to confirm the pattern (an SSO app must redirect to a DNS record), then scaffolds it correctly. Staging reaches parity.

Recursive CTE: acme-prod-admin-sso transitive dependencies

Ran the project's downstream CTE via db_query:

Depth	From	Relationship	To	Type
1	acme-prod-admin-sso	uses	SsoApplication	Library
1	acme-prod-admin-sso	redirects_to	acme-prod-admin-dns	Cloudflare
2	acme-prod-admin-dns	uses	DnsRecord	Library

Transitive closure: **SsoApplication → acme-prod-admin-dns → DnsRecord.**

SSO must redirect to a DnsRecord

Confirmed in the graph and conventions:

- Production has acme-prod-admin-sso **redirects_to** acme-prod-admin-dns, and that DNS record **uses** DnsRecord.
- SsoApplication is documented as requiring redirect URIs on the matching DnsRecord hostname (libs/sso-app, configs/claude-code/CLAUDE.md).

So the SSO app does not redirect to a raw hostname in isolation — it redirects to the **admin DNS component**, which composes **DnsRecord.**

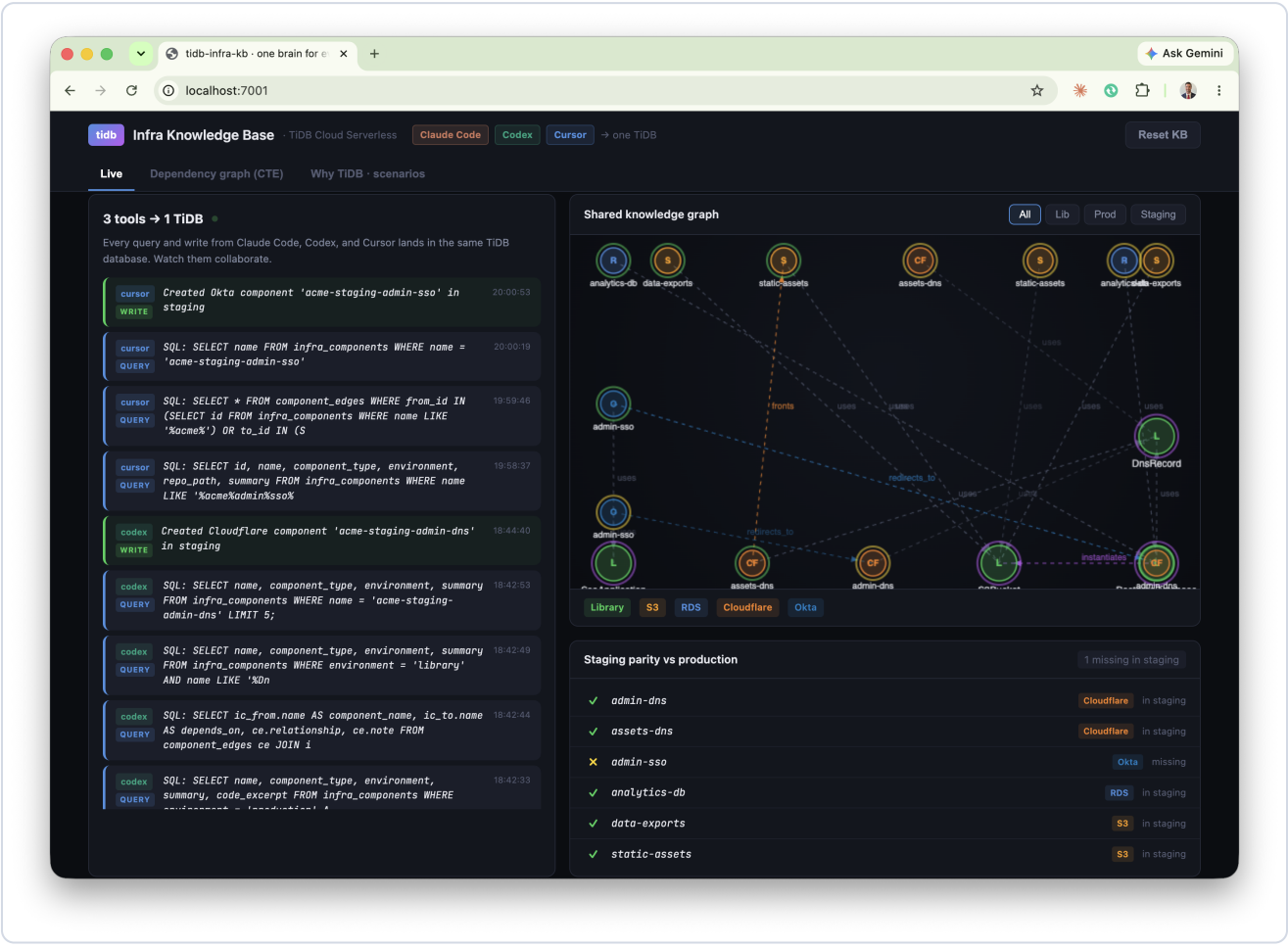
Staging scaffold + KB writeback

1. **environments/staging/dns.ts** — added acme-staging-admin-dns and adminDnsHostname (needed for the redirect URI).
2. **environments/staging/sso.ts** — new SsoApplication with redirect https://${adminDnsHostname}/callback.
3. **write_component** — recorded acme-staging-admin-sso (Okta, staging, composes SsoApplication).
4. **db_execute** — edge acme-staging-admin-sso **redirects_to** acme-staging-admin-dns.

KB edges for staging SSO:

- uses → SsoApplication
- redirects_to → acme-staging-admin-dns

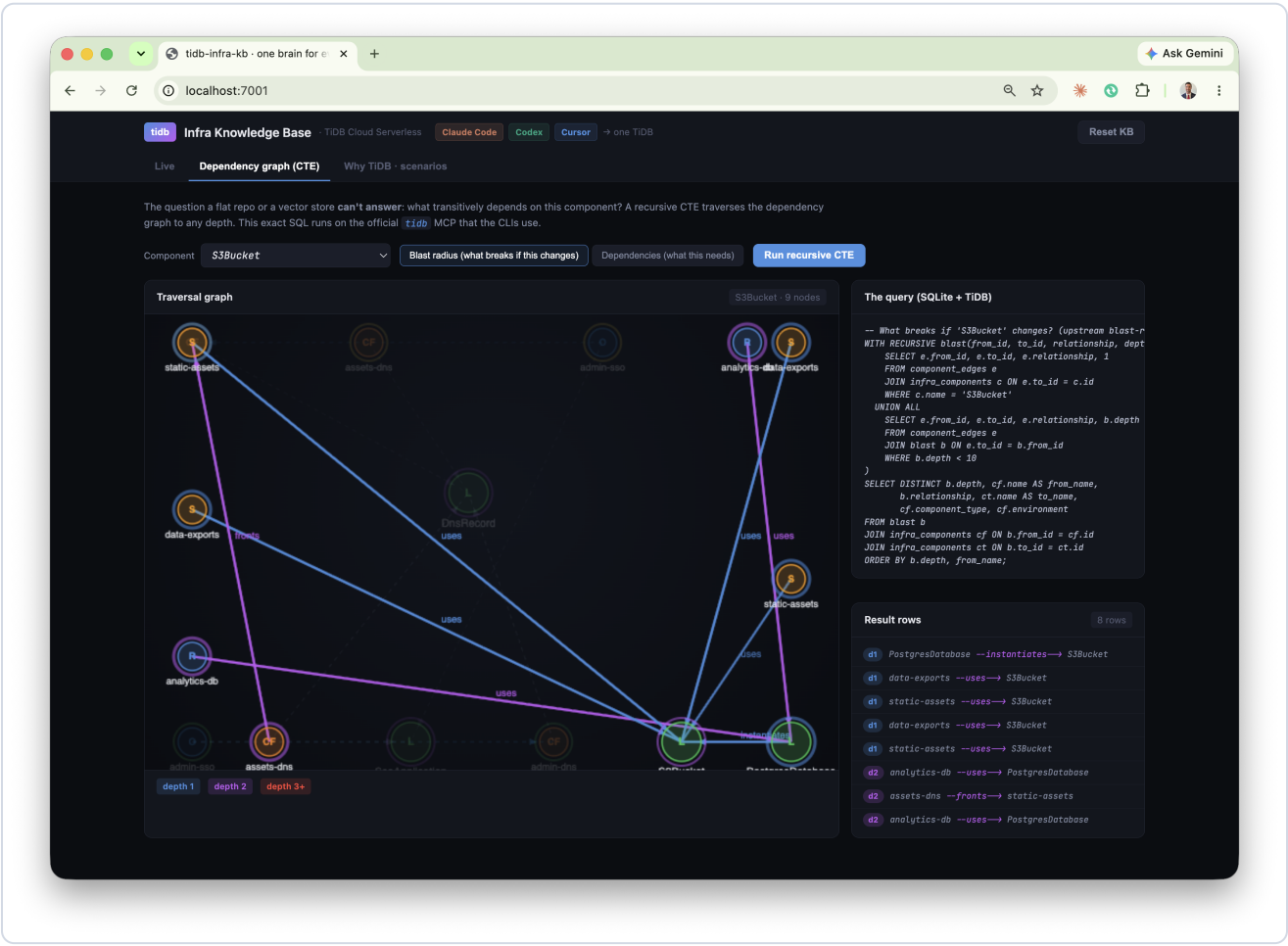
Real Cursor output - the headline moment. The dependency traversal returns the chain `SsoApplication → acme-prod-admin-dns → DnsRecord`, proving the pattern before writing a line of code.



The result, in the dashboard: all three tools' activity in one feed (cursor, codex, claude-code) and the staging-parity strip now fully green. Built by three tools sharing one memory.

The query a vector database can't answer

The headline capability: **graph reachability**. "What breaks if I change this component?" requires traversing dependency edges to arbitrary depth - a transitive closure, not a similarity ranking. mem9 stores every component and its typed edges as a graph and returns that multi-hop traversal through the API.



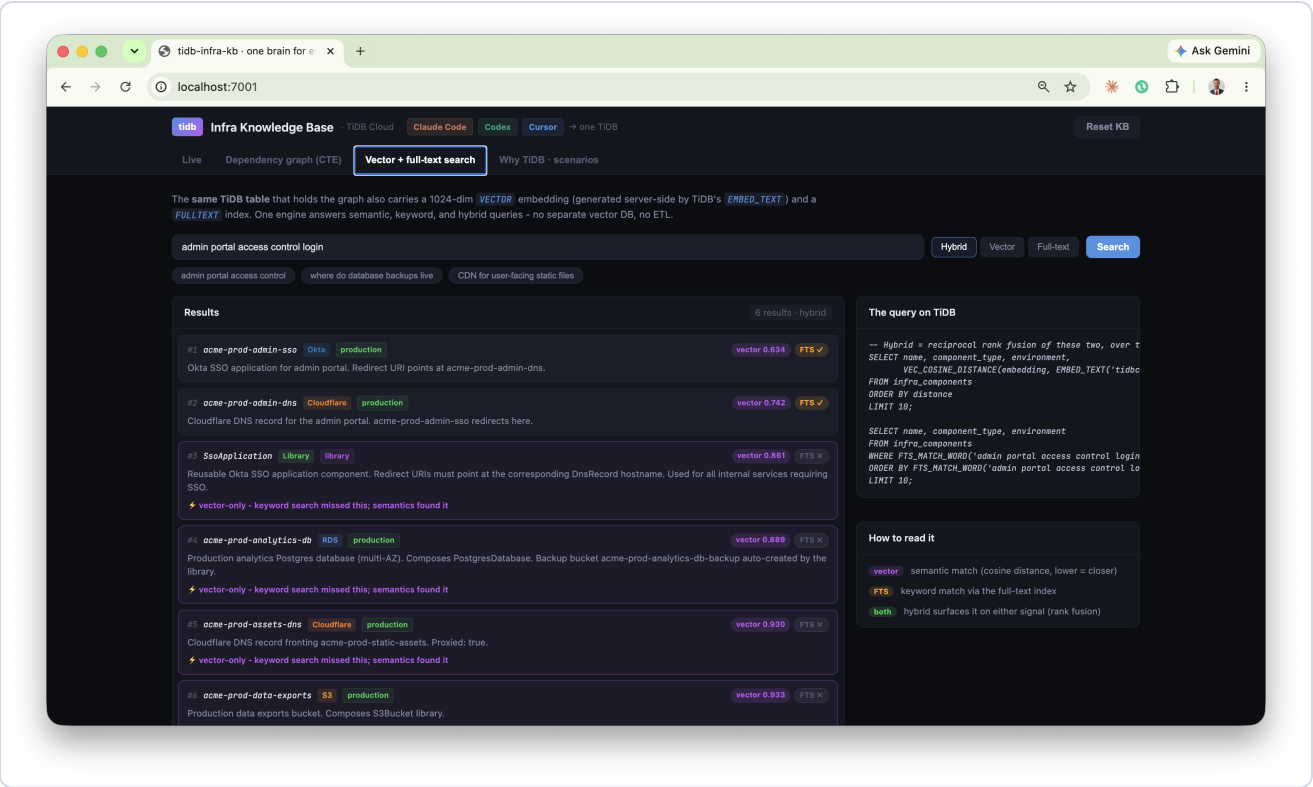
*Blast radius of the `S3Bucket` library on the live dashboard, visualised as a depth-coloured subgraph with the traversal result beside it. It reaches the RDS databases **two hops away** - their backup buckets compose `S3Bucket` - a connection no text search could surface.*

```sql
-- Graph reachability: everything that breaks if S3Bucket changes.
-- mem9 walks the dependency graph and returns the transitive set;
-- in relational terms the traversal is a recursive CTE:
WITH RECURSIVE blast(from_id, to_id, relationship, depth) AS (
    SELECT from_id, to_id, relationship, 1
    FROM edges WHERE to_component = 'S3Bucket'
  UNION ALL
    SELECT e.from_id, e.to_id, e.relationship, b.depth + 1
    FROM edges e JOIN blast b ON e.to_id = b.from_id
    WHERE b.depth < 10
)
SELECT depth, from_id, relationship, to_id FROM blast ORDER BY depth;
```

Why a vector store can't do this. Vector similarity finds *related-looking* text. It cannot compute a transitive closure over a dependency graph - that is reachability, a different operation. mem9 keeps the graph in the same store as the components and their embeddings, so the agent gets similarity *and* reachability from one memory, through one API - no second system to reconcile.

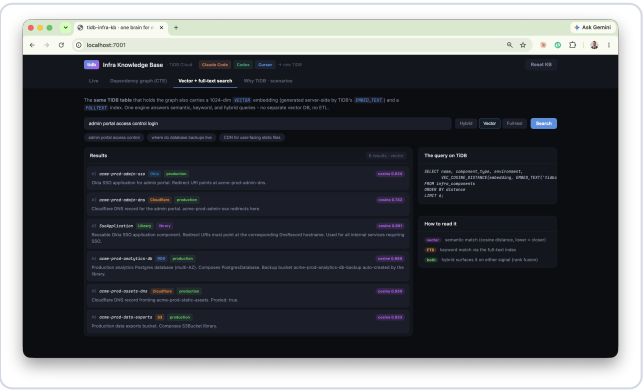
One engine: vector + full-text + graph, one memory

Every component mem9 stores carries a server-side embedding (1024-dim, generated on write via TiDB's `EMBED_TEXT`). A single `query_knowledge_base` call returns **hybrid recall that blends semantic (embedding) and keyword matching**, over the same memory that holds the dependency graph. No separate vector database, no ETL, no sync lag.

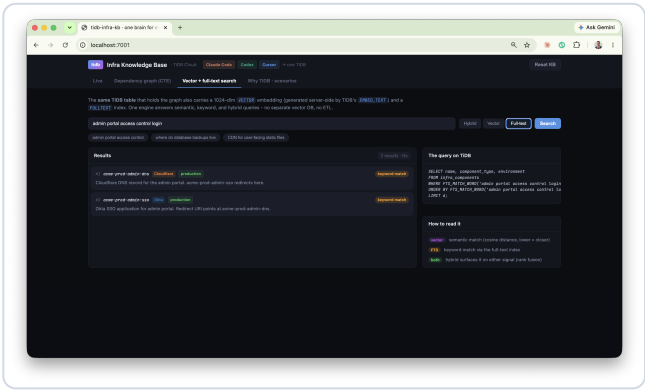


mem9 hybrid recall for "admin portal access control login", blending semantic and keyword matching over one memory. In this capture, the top rows match on both signals; the rest are surfaced by the embedding - components a keyword search alone would miss.

> **The headline:** keyword matching alone can miss semantically-related components; hybrid recall catches them. In this capture, a keyword-only query surfaced **2** components where hybrid surfaced **6** - including `SsoApplication` and the analytics database, which contain none of the query's keywords. One call gives you both, ranked together.



The semantic signal: ranking by embedding similarity (e.g. TiDB's `VEC_COSINE_DISTANCE` over `EMBED_TEXT`). Catches meaning, not just words.



The keyword signal: full-text matching over the same components. Precise on exact terms, but blind to synonyms - so it misses the rest.

> **Why it matters:** a coding agent searching its memory for "how do we do auth" needs *meaning*, not string matching. mem9 gives the agent vector recall, keyword precision, and graph reachability through one API - over the same memory that already holds the structured state.

Why TiDB is the right brain for agents

In the emerging pattern, the database is the agent's persistent memory, execution-state store, and shared knowledge base. That demands more than a vector index.

One engine, every access pattern

Agents need **structured state**, **graph traversal** for relationships, **vector search** for semantic memory, and **full-text (BM25)** for knowledge - all over the same data, with no ETL and no sync lag. TiDB provides all of them in one MySQL-compatible engine, and mem9 exposes them through a single API: hybrid recall (vector + full-text) and multi-hop graph reachability over the same memory, with the embedding generated server-side by *EMBED_TEXT* on write.

Durable agent state

Every write records the component and its dependency edges into the shared memory. TiDB's distributed ACID underpins mem9's durability, so team state doesn't fragment across the partial-write failures that make multi-agent systems hard to debug.

Built for agent scale

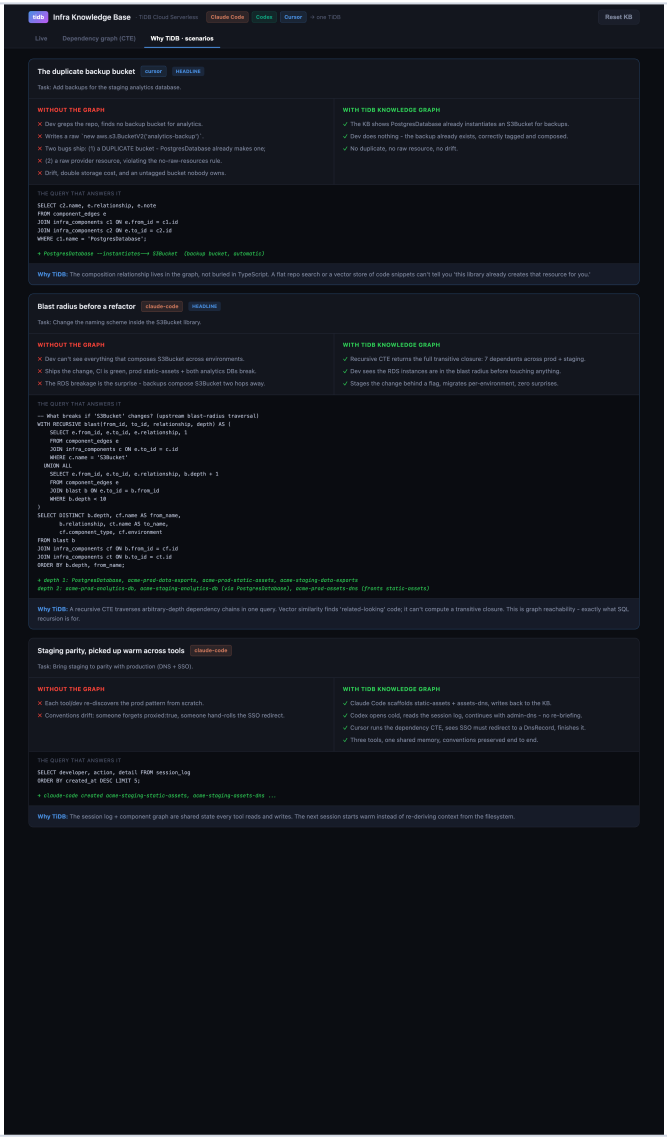
TiDB Cloud scales to zero with pay-per-request pricing. Millisecond database branching gives each agent or workspace an isolated environment - the pattern behind production fleets running millions of agent databases on TiDB Cloud.

How it compares for this workload

CAPABILITY	TIDB CLOUD	VECTOR DB (PINECONE)	POSTGRES / PGVECTOR
Recursive graph traversal (CTE)	✓	✗	✓ (single node)
SQL + JOINs + ACID	✓	✗	✓
Vector + full-text in one engine	✓	vector only	partial
Horizontal scale / millions of tenants	✓	partial	✗

Concrete scenarios the graph prevents

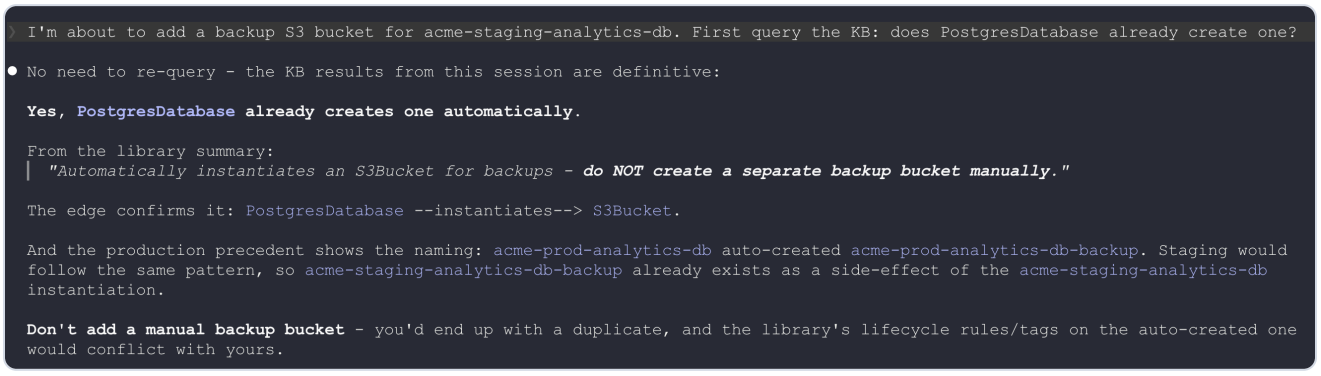
Two everyday infrastructure mistakes, and how a queryable graph stops them before code is written.



In-product scenario cards. Each contrasts the failure without the graph against the corrected path with TiDB, and shows the exact query that answers it.

The duplicate-bucket guardrail, caught live

Asked to add a backup bucket for the staging database, a coding tool queried the graph first - and stopped itself:



Real tool output: it found `PostgresDatabase --instantiates⟶ S3Bucket` in the graph and concluded "Don't add a manual backup bucket - you'd end up with a duplicate, and the library's lifecycle rules/tags on the auto-created one would conflict with yours." The duplicate was never written.

The duplicate backup bucket. Asked to "add backups" for a database, a tool greps, finds nothing, and creates a raw duplicate. The graph shows *`PostgresDatabase`* already *instantiates* a backup bucket - so the right action is to write nothing.

Blast radius before a refactor. Before changing the *`S3Bucket`* library, one graph-reachability query returns the full set of dependents across every environment - including the RDS instances two hops away. The change is staged safely instead of breaking production.

How the tools are driven: the prompts

The behaviour is prompt-driven. Two layers: a **standing instruction** in each tool's rules file (always read/write the shared memory), and the **per-task prompt** the developer types. Nothing is hard-coded - this is how a real team would configure it.

Standing instruction (rules file: CLAUDE.md / AGENTS.md / .cursorrules)

```
Before writing any infrastructure code, query the infra-kb MCP first
(what exists, how it connects, what previous sessions did). Never use raw
aws.* / cloudflare.* resources - compose the library components. After you
create anything, record it with write_component so the next session starts warm.
```

Pane 1 - Claude Code

```
Use the infra-kb MCP. Query the knowledge base: what components exist in
staging vs production, and what is staging missing? Then scaffold the missing
staging static-assets bucket and its Cloudflare DNS record, composing the
S3Bucket and DnsRecord libraries - never raw aws.* resources. Follow the
acme-<env>-<name> naming and required tags. Record each new component with
write_component. Pass developer='claude-code' on every infra-kb call
so the activity feed attributes the work.
```

Pane 2 - Codex seat (opens cold, picks up warm; also runs on MiniMax-M3)

```
Use the infra-kb MCP. Query the shared memory - what did the previous
session just create? Continue the staging build: add the admin-portal DNS record
(acme-staging-admin-dns) matching the production pattern, composing DnsRecord.
Record it with write_component. Pass developer='codex' on every infra-kb
call so the activity feed attributes the work.
```

Pane 3 - Cursor (check the dependency chain, then build)

```
Use the infra-kb MCP. Query the transitive dependencies of acme-prod-admin-sso -
what does it depend on, and does an SSO app redirect to a DnsRecord? Then scaffold
acme-staging-admin-sso composing SsoApplication, with the redirect URI pointing at
acme-staging-admin-dns. Record it with write_component (developer='cursor')
so the activity feed attributes the work.
```

The guardrail prompt (duplicate-bucket scenario)

```
I'm about to add a backup S3 bucket for acme-staging-analytics-db. First query
the KB: does PostgresDatabase already create one?
```

Every call the demo makes

The agents use just two MCP tools on the *infra-kb* server - *query_knowledge_base* (natural language) and *write_component*. mem9 turns those into hybrid recall, graph traversal, and durable writes on TiDB Cloud. Below: the calls the tools make (left), and what mem9 runs on TiDB underneath (right). Nothing is mocked.

1. Inspect - what exists where (Claude Code)

```
# MCP call -> infra-kb.query_knowledge_base
{ "query": "what components exist in staging vs production; what is staging missing?",
  "developer": "claude-code" }
# mem9 returns the matching components from appId acme_pulumi_kb via hybrid recall.
```

2. Pick up warm - read the shared memory (Codex / MiniMax)

```
# MCP call -> infra-kb.query_knowledge_base
{ "query": "what did the previous session just create?",
  "developer": "codex" }
# mem9 surfaces the recent logged activity + newly-written components. No re-briefing.
```

3. The write - what write_component records (Cursor)

```
# MCP call -> infra-kb.write_component
{ "name": "acme-staging-admin-sso", "type": "Okta", "env": "staging",
  "summary": "...", "depends_on": "SsoApplication",
  "account_ref": "prod", "developer": "cursor" }
# mem9 stores a memory in acme_pulumi_kb: content + metadata (type/env/depends_on),
# embeds it server-side with EMBED_TEXT, and logs the action - all in one space.
```

4. Graph reachability - transitive dependencies & blast radius (Dependency tab)

```
-- mem9 walks the dependency graph and returns the transitive set.
-- In relational terms, the traversal is a recursive CTE on the edges:
WITH RECURSIVE deps(from_id, to_id, relationship, depth) AS (
    SELECT from_id, to_id, relationship, 1
    FROM edges WHERE from_component = 'acme-prod-admin-sso'
  UNION ALL
    SELECT e.from_id, e.to_id, e.relationship, d.depth + 1
    FROM edges e JOIN deps d ON e.from_id = d.to_id
    WHERE d.depth < 10
)
SELECT depth, from_id, relationship, to_id FROM deps ORDER BY depth;
-- reverse the edge direction for blast radius (everything that breaks if X changes).
```

5. Hybrid recall - one call, two signals (Search tab)

```
# MCP call -> infra-kb.query_knowledge_base { "query": "admin portal access control" }
# mem9 blends semantic + keyword recall over the same memory - the hybrid
```

```
# search TiDB is built for (embedding similarity + full-text) — and returns
# one ranked result set. In our test run, keyword alone surfaced 2; hybrid, 6.
```

Running it on your own repo

Everything you saw - query-before-build, convention checks, transitive dependency traversal, hybrid search, cross-tool handoff - is already general. The one piece to add for a real codebase is a small **ingestion job** that populates the mem9 graph from your source. Everything downstream is unchanged; it reads the same shared memory.

The ingestion job (runs in CI on every merge)

1. **Read the resource graph.** Parse the Pulumi program (*`pulumi stack export`* / the resource tree, or the TypeScript AST) to enumerate every component and its parent-child and cross-references.
2. **Write components + edges.** Each component is recorded with *`write_component`*; each relationship rides in its metadata as a typed edge (*`instantiates`*, *`fronts`*, *`redirects_to`*, ...).
3. **Embed.** mem9 embeds each component server-side (*`EMBED_TEXT`*) on write, so semantic search works the moment it lands - no embedding pipeline to run.
4. **Log the scan** so tools can see "graph last synced at commit abc123."

```
# src/ingest.py — run in CI; keeps the mem9 graph in sync with main
for resource in pulumi_resource_graph():
    write_component(repo="pulumi", name=resource.urn_name, type=resource.kind,
                    env=resource.stack, summary=resource.doc,
                    depends_on=resource.parent)   # mem9 embeds + records the edge
```

> **That is the whole gap between this demo and production.** The graph the tools query is the same shape; only its *source* changes from the seed script to your live repo. Query, graph traversal, search, convention-enforcement, and cross-tool memory all work as shown.

How to run the demo

The repository ships everything: the ingestion job, the MCP servers, per-tool configs, the dashboard, and a presenter's runbook.

1. **Configure mem9.** `cp .env.example .env` and paste your `MEM9_API_KEY` (one key scopes one team's space). `MEM9_BASE_URL` defaults to `https://api.mem9.ai`.
2. **Set up.** `./setup.sh` - installs dependencies, ingests the seed graph into mem9 (`python -m src.ingest`), and generates MCP configs for Claude Code, Codex, and Cursor.
3. **Launch the dashboard.** `./demo.sh` → open `http://localhost:7001` (the screen you narrate).
4. **Open three terminals.** In each, run `claude`, `minimax -m MiniMax-M3` (the Codex seat - any MCP-capable agent CLI works, including `codex` where available), and `cursor-agent` from the repo. All three connect to the shared mem9 memory over MCP automatically.
5. **Follow the prompts** in `DEMO.md` / `PRESENTER.md`, or use `RECORDING.md` for the full runbook with a narration script. As the tools work, the dashboard updates live from mem9.

The MCP servers each tool connects to (one per repo namespace)

SERVER	NAMESPACE (APPID)	WHAT IT PROVIDES
`infra-kb-pulumi`	`acme_pulumi_kb`	`query_knowledge_base` + `write_component` - hybrid recall, graph reachability, convention-checked writes
`infra-kb-lza`	`acme_lza_kb`	Same tools, scoped to the Landing-Zone repo. Routing is explicit (`MEM9_REPO`), never inferred.

Each server is the same `src.mcp_server` process pinned to one repo via `MEM9_REPO`; both call the mem9 REST API, which manages the TiDB Cloud cluster underneath. Agents hold only the API key - never database credentials.

> **Repository:** `github.com/stephenlthorn/mem9-ai-coding` · Apache-2.0. mem9 runs fully managed on TiDB Cloud; a self-hosted deployment is also supported.

Query before you build. Prove the blast radius.
Hand off warm to the next tool.